

2. Modulo

Modulo hiểu đơn giản là phần dư khi chia một số cho một số khác.

Ký hiệu:

$$a \bmod m$$

nghĩa là phần dư của a khi chia cho m .

Ví dụ:

$$\begin{aligned} 17 \bmod 5 &= 2 \\ 23 \bmod 7 &= 2 \\ 100 \bmod 10 &= 0 \end{aligned}$$

Trong lập trình, modulo thường dùng để tránh số quá lớn, đặc biệt trong các bài đếm số cách, DP, tổ hợp, lũy thừa,...

Các modulo hay gặp:

$$\begin{aligned} 10^9 + 7 \\ 998244353 \end{aligned}$$

Hai số này đều là số nguyên tố.

Một cách nhìn khác của modulo:

$$a \bmod b = a - b \times \left\lfloor \frac{a}{b} \right\rfloor$$

Ví dụ:

$$\begin{aligned} 17 \bmod 5 &= 17 - 5 * \text{floor}(17 / 5) \\ &= 17 - 5 * 3 \\ &= 2 \end{aligned}$$

Công thức này sẽ liên hệ nhiều với phần **số học** phía trước.

Điểm quan trọng là giá trị:

$$\left\lfloor \frac{n}{i} \right\rfloor$$

khi i chạy từ 1 đến n ta thấy có nhiều đoạn liên tiếp mà giá trị trên bằng nhau. Ví dụ với $n = 10$:

i	:	1	2	3	4	5	6	7	8	9	10
$\text{floor}(10/i)$:	10	5	3	2	2	1	1	1	1	1	1

Vì vậy trong nhiều bài toán có dạng:

$$a \bmod i = a - i \times \left\lfloor \frac{a}{i} \right\rfloor$$

ta có thể nhóm các i có cùng giá trị:

$$\left\lfloor \frac{a}{i} \right\rfloor$$

lại thành một đoạn liên tiếp, rồi tính nhanh tổng trên đoạn đó.

Kỹ thuật này giúp nhiều bài toán số học giảm từ $O(n)$ xuống khoảng $O(\sqrt{n})$. Vì bọn e thấy bản chất của $\left\lfloor \frac{n}{i} \right\rfloor$ thật ra là xét 2 TH nếu mà $i < \sqrt{n}$ thì sẽ có tối đa $\sqrt{n}$ giá trị khác nhau (tưởng tượng mỗi i sẽ ứng với một giá trị mà chỉ có tối đa $\sqrt{n}$ giá trị i mà thôi) còn nếu mà $i > \sqrt{n}$ thì $\left\lfloor \frac{n}{i} \right\rfloor$ sẽ bị chặn bởi 0 và $\sqrt{n}$ nên cũng chỉ có tối đa $\sqrt{n}$ số mà thôi. Vậy tổng tối đa chỉ có nhiều nhất là $2\sqrt{n}$ giá trị phân biệt.

Một tính chất hay khác của phép modulo là: nếu ta liên tục lấy phần dư như trong thuật toán Euclid:

$$a \leftarrow a \bmod b$$

thì giá trị sẽ giảm rất nhanh.

Cụ thể, sau một vài bước, số hiện tại thường giảm ít nhất khoảng một nửa. Vì vậy số lần thực hiện phép modulo cho đến khi về 0 chỉ khoảng:

$$O(\log a)$$

Đây là lý do thuật toán Euclid tính $\text{gcd}(a, b)$ rất nhanh, dù a và b có thể rất lớn.

1. Tính chất cơ bản

Với mọi số nguyên a, b và số nguyên dương m :

$$(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$$

$$(a - b) \bmod m = ((a \bmod m) - (b \bmod m)) \bmod m$$

$$(a \times b) \bmod m = ((a \bmod m) \times (b \bmod m)) \bmod m$$

Nói ngắn gọn, modulo xử lý tốt với:

- cộng
- trừ
- nhân

Code thường dùng:

```

const int mod = 1e9 + 7;

void add(int &x, int y) {
    x += y;
    if(x >= mod) x -= mod;
}

void sub(int &x, int y) {
    x -= y;
    if(x < 0) x += mod;
}

int mul(int x, int y) {
    return x * y % mod;
}

```

Nếu tích có thể bị tràn `long long`, dùng `__int128`:

```

long long mul(long long x, long long y, long long mod) {
    return (__int128)x * y % mod;
}

```

2. Modulo với số âm

Trong C++, kết quả `%` với số âm có thể âm.

Ví dụ:

```
cout << -1 % 5;
```

Kết quả là:

```
-1
```

Nhưng trong toán học, ta thường muốn phần dư nằm trong đoạn:

$$0 \leq r < m$$

Nên:

$$-1 \equiv 4 \pmod{5}$$

Code chuẩn hóa:

```
int norm(int x) {  
    x %= mod;  
    if(x < 0) x += mod;  
    return x;  
}
```

Hoặc nếu x có thể âm rất lớn:

```
int norm(int x) {  
    return (x % mod + mod) % mod;  
}
```

3. Đồng dư modulo

Ta nói:

$$a \equiv b \pmod{m}$$

nếu a và b có cùng phần dư khi chia cho m .

Tương đương:

$$m \mid (a - b)$$

Ví dụ:

$$17 \equiv 2 \pmod{5}$$

vì:

$$17 - 2 = 15$$

và 15 chia hết cho 5.

Một số tính chất:

Nếu:

$$a \equiv b \pmod{m}$$

và:

$$c \equiv d \pmod{m}$$

thì:

$$a + c \equiv b + d \pmod{m}$$

$$a - c \equiv b - d \pmod{m}$$

$$a \times c \equiv b \times d \pmod{m}$$

Đặc biệt:

$$a \equiv b \pmod{m}$$

thì:

$$a \times c \equiv b \times c \pmod{m}$$

4. Không được tùy tiện chia hai vế

Trong số thực, nếu:

$$a \times c = b \times c$$

và $c \neq 0$, ta chia hai vế cho c được.

Nhưng trong modulo thì không được làm tùy tiện.

Ví dụ:

$$2 \times 1 \equiv 2 \times 4 \pmod{6}$$

vì:

$$2 \equiv 8 \pmod{6}$$

Nhưng không thể chia hai vế cho 2 để suy ra:

$$1 \equiv 4 \pmod{6}$$

vì điều này sai.

Lý do là 2 không có nghịch đảo modulo 6:

$$\gcd(2, 6) = 2 \neq 1$$

Kết luận:

Nếu:

$$a \times c \equiv b \times c \pmod{m}$$

thì chỉ được chia hai vế cho c khi:

$$\gcd(c, m) = 1$$

Khi đó:

$$a \equiv b \pmod{m}$$

Nếu $\gcd(c, m) \neq 1$ thì không được chia trực tiếp.

5. Lũy thừa nhanh modulo

Bài toán:

Tính:

$$a^n \bmod m$$

Nếu nhân từng lần thì độ phức tạp là:

$$O(n)$$

Ta dùng lũy thừa nhị phân để đưa về:

$$O(\log n)$$

Ý tưởng:

Nếu n chẵn:

$$a^n = a^{n/2} \times a^{n/2}$$

Nếu n lẻ:

$$a^n = a \times a^{n-1}$$

Code:

```
int Pow(int a, int b, int mod) {  
    int res = 1;  
    a %= mod;  
  
    while(b > 0) {  
        if(b & 1) res = res * a % mod;  
        a = a * a % mod;  
        b >>= 1;  
    }  
  
    return res;  
}
```

Ví dụ:

$$3^{13} = 3^8 \times 3^4 \times 3^1$$

vì:

$$13 = 8 + 4 + 1$$

6. Tính modulo của một chuỗi số rất dài

Bài toán:

Cho một số rất lớn dưới dạng xâu:

123456789123456789123456789

Cần tính số đó modulo m .

Vì số quá lớn nên không thể lưu bằng `long long`.

Ý tưởng là đọc từng chữ số từ trái sang phải.

Giả sử phần đã đọc đang có giá trị là x .

Khi thêm chữ số d vào cuối, số mới là:

$$x \times 10 + d$$

Vậy phần dư mới là:

$$rem = (rem \times 10 + d) \bmod m$$

Code:

```
int mod_string(string s, int m) {  
    int rem = 0;  
  
    for(char c : s) {  
        int d = c - '0';  
        rem = (rem * 10 + d) % m;  
    }  
  
    return rem;  
}
```

Ví dụ tính $1234 \bmod 7$:

rem = 0

Đọc 1: rem = 1

Đọc 2: rem = $12 \bmod 7 = 5$

Đọc 3: rem = $53 \bmod 7 = 4$

Đọc 4: rem = $44 \bmod 7 = 2$

Vậy:

$$1234 \bmod 7 = 2$$

Tổng quát nếu xâu ở hệ cơ số `base`:

$$rem = (rem \times base + digit) \bmod m$$

7. Nghịch đảo modulo

Nghịch đảo modulo của a theo modulo m là số x thỏa:

$$a \times x \equiv 1 \pmod{m}$$

Khi đó:

$$x = a^{-1}$$

Ví dụ:

$$2^{-1} \pmod{5} = 3$$

vì:

$$2 \times 3 = 6 \equiv 1 \pmod{5}$$

Nghịch đảo của a modulo m tồn tại khi và chỉ khi:

$$\gcd(a, m) = 1$$

Ví dụ:

2 có nghịch đảo modulo 5 vì $\gcd(2, 5) = 1$
2 không có nghịch đảo modulo 6 vì $\gcd(2, 6) = 2$

8. Chia trong modulo

Trong modulo, không chia trực tiếp được.

Sai:

```
ans = a / b % mod;
```

Muốn chia cho b , ta phải nhân với nghịch đảo của b :

$$\frac{a}{b} \equiv a \times b^{-1} \pmod{m}$$

Tức là:

```
ans = a * inv(b) % mod;
```

Nhưng cách này chỉ đúng khi b có nghịch đảo modulo m .

Tức là:

$$\gcd(b, m) = 1$$

9. Định lý Fermat nhỏ

Nếu p là số nguyên tố và a không chia hết cho p , thì:

$$a^{p-1} \equiv 1 \pmod{p}$$

Suy ra:

$$a^{p-2} \times a \equiv 1 \pmod{p}$$

Vậy:

$$a^{-1} \equiv a^{p-2} \pmod{p}$$

Nếu mod là số nguyên tố:

```
int inv(int a) {  
    return Pow(a, mod - 2, mod);  
}
```

Ví dụ:

```
mod = 1e9 + 7  
inv(2) = 500000004
```

vì:

$$2 \times 500000004 \equiv 1 \pmod{10^9 + 7}$$

10. Tính nhiều nghịch đảo modulo

Nếu cần tính nhiều nghịch đảo từ 1 đến n , không nên gọi **Pow** quá nhiều lần.

Với mod nguyên tố:

$$\text{inv}[i] = \text{mod} - \left\lfloor \frac{\text{mod}}{i} \right\rfloor \times \text{inv}[\text{mod} \bmod i] \bmod \text{mod}$$

Code:

```
int inv[N];  
  
void build_inv(int n) {
```

```

    inv[1] = 1;

    for(int i = 2; i <= n; i++) {
        inv[i] = mod - mod / i * inv[mod % i] % mod;
    }
}

```

Lưu ý công thức này dùng khi:

$$1 \leq i < mod$$

và `mod` là số nguyên tố.

11. Tổ hợp modulo

Ta có:

$$C(n, k) = \frac{n!}{k!(n-k)!}$$

Nếu `mod` là số nguyên tố:

$$C(n, k) \equiv fact[n] \times invfact[k] \times invfact[n - k] \pmod{mod}$$

Code:

```

int fact[N], invfact[N];

void build_fact(int n) {
    fact[0] = 1;

    for(int i = 1; i <= n; i++) {
        fact[i] = fact[i - 1] * i % mod;
    }

    invfact[n] = Pow(fact[n], mod - 2, mod);

    for(int i = n - 1; i >= 0; i--) {
        invfact[i] = invfact[i + 1] * (i + 1) % mod;
    }
}

int C(int n, int k) {
    if(k < 0 || k > n) return 0;
    return fact[n] * invfact[k] % mod * invfact[n - k] % mod;
}

```

Độ phức tạp chuẩn bị:

$$O(n + \log mod)$$

Mỗi truy vấn:

$$O(1)$$

12. Nếu modulo không nguyên tố thì xử lý thế nào?

Nếu **mod** không nguyên tố thì không được dùng bừa:

$$a^{-1} = a^{mod-2}$$

Công thức Fermat chỉ đúng khi modulo là số nguyên tố.

Với modulo không nguyên tố, có hai hướng xử lý hay gặp.

Cách 1. Phân tích thừa số nguyên tố rồi cộng trừ số mũ

Cách này dùng khi biểu thức có dạng tích chia, nhưng kết quả cuối cùng chắc chắn là số nguyên.

Ví dụ:

$$\frac{12 \times 10}{6}$$

Ta phân tích:

$$12 = 2^2 \times 3$$

$$10 = 2 \times 5$$

$$6 = 2 \times 3$$

Khi nhân thì cộng số mũ, khi chia thì trừ số mũ:

$$\begin{aligned}\frac{12 \times 10}{6} &= 2^{2+1-1} \times 3^{1-1} \times 5^1 \\ &= 2^2 \times 5 = 20\end{aligned}$$

Sau đó mới lấy modulo.

Code ý tưởng:

```
map<int, int> cnt;

void factor(int x, int sign) {
    for(int i = 2; i <= x / i; i++) {
        while(x % i == 0) {
            cnt[i] += sign;
            x /= i;
        }
    }
}
```

```
    if(x > 1) cnt[x] += sign;
}
```

Trong đó:

```
sign = 1  nếu số nằm ở tử
sign = -1 nếu số nằm ở mẫu
```

Sau đó nhân lại:

```
int calc(int mod) {
    int ans = 1;

    for(auto [p, e] : cnt) {
        ans = ans * Pow(p, e, mod) % mod;
    }

    return ans;
}
```

Cách này hay dùng cho:

- tổ hợp $C(n, k)$ với modulo không nguyên tố
- biểu thức có chia nhưng kết quả vẫn là số nguyên
- giai thừa chia giai thừa

Cách 2. Dùng Extended Euclid hoặc phi Euler (Nâng cao)

Cách này dùng khi thật sự cần tìm nghịch đảo của a modulo m .

Điều kiện vẫn là:

$$\gcd(a, m) = 1$$

Nếu điều kiện đúng thì nghịch đảo tồn tại.

Extended Euclid

Ta cần tìm x sao cho:

$$a \times x \equiv 1 \pmod{m}$$

Tương đương:

$$a \times x + m \times y = 1$$

Code:

```

int exgcd(int a, int b, int &x, int &y) {
    if(b == 0) {
        x = 1;
        y = 0;
        return a;
    }

    int x1, y1;
    int g = exgcd(b, a % b, x1, y1);

    x = y1;
    y = x1 - a / b * y1;

    return g;
}

int inv_any_mod(int a, int mod) {
    int x, y;
    int g = exgcd(a, mod, x, y);

    if(g != 1) return -1;

    x %= mod;
    if(x < 0) x += mod;

    return x;
}

```

Nếu trả về -1 nghĩa là không tồn tại nghịch đảo.

Phi Euler

Nếu:

$$\gcd(a, m) = 1$$

thì:

$$a^{\varphi(m)} \equiv 1 \pmod{m}$$

Suy ra:

$$a^{-1} \equiv a^{\varphi(m)-1} \pmod{m}$$

Code:

```

int phi(int n) {
    int ans = n;

    for(int i = 2; i <= n / i; i++) {

```

```

        if(n % i == 0) {
            ans = ans / i * (i - 1);

            while(n % i == 0) {
                n /= i;
            }
        }

        if(n > 1) {
            ans = ans / n * (n - 1);
        }

        return ans;
    }

    int inv_by_phi(int a, int mod) {
        if(__gcd(a, mod) != 1) return -1;
        return Pow(a, phi(mod) - 1, mod);
    }

```

Tóm lại:

mod nguyên tố:
dùng Fermat

mod không nguyên tố, $\gcd(a, \text{mod}) = 1$:
dùng Extended Euclid hoặc phi Euler

mod không nguyên tố, $\gcd(a, \text{mod}) \neq 1$:
không có nghịch đảo, không được chia trực tiếp

biểu thức có chia nhưng kết quả chắc chắn nguyên:
phân tích thừa số nguyên tố rồi cộng trừ số mũ

13. Một số ứng dụng hay gặp

Lấy chữ số

Chữ số cuối của n :

$$n \bmod 10$$

Bỏ chữ số cuối:

$$\left\lfloor \frac{n}{10} \right\rfloor$$

Code:

```
int digit = n % 10;  
n /= 10;
```

Kiểm tra chẵn lẻ

Nếu:

$$n \bmod 2 = 0$$

thì n chẵn.

Nếu:

$$n \bmod 2 = 1$$

thì n lẻ.

Code:

```
if(n % 2 == 0) {  
    // n chẵn  
}  
else {  
    // n lẻ  
}
```

Đếm cặp theo phần dư

Bài toán:

Đếm số cặp (i, j) sao cho:

$$(a_i + a_j) \bmod m = 0$$

Nếu:

$$a_i \bmod m = r$$

thì cần:

$$a_j \bmod m = (m - r) \bmod m$$

Code ý tưởng:

```
int ans = 0;  
for(int i = 1; i <= n; i++) {
```

```
int r = a[i] % m;  
int need = (m - r) % m;  
  
ans += cnt[need];  
cnt[r]++;  
}
```

Dạng này rất hay gặp trong các bài đếm cặp, prefix sum modulo.

14. Lỗi hay gặp

Quên modulo sau khi nhân

Sai:

```
ans = ans * x;
```

Đúng:

```
ans = ans * x % mod;
```

Trừ bị âm

Sai:

```
ans = (a - b) % mod;
```

Đúng:

```
ans = (a - b + mod) % mod;
```

Chia trực tiếp trong modulo

Sai:

```
ans = a / b % mod;
```

Đúng nếu b có nghịch đảo:


```
ans = a * inv(b) % mod;
```

Dùng Fermat khi modulo không nguyên tố

Sai nếu `mod` không nguyên tố:

```
inv = Pow(a, mod - 2, mod);
```

Cần kiểm tra:

```
gcd(a, mod) == 1
```

hoặc dùng Extended Euclid hoặc phi Euler.

15. Code mẫu tổng hợp

```
#include<bits/stdc++.h>
#define int long long
#define oo 1000000000000000000
#define ii pair<int, int>
#define fi first
#define se second
#define all(x) x.begin(), x.end()
#define uniq(x) x.erase(unique(all(x)), x.end())
#define pb push_back
using namespace std;

const int mod = 1e9 + 7;

void add(int &x, int y) {
    x += y;
    if(x >= mod) x -= mod;
}

void sub(int &x, int y) {
    x -= y;
    if(x < 0) x += mod;
}

int Pow(int a, int b, int mod) {
    int res = 1;
    a %= mod;

    while(b > 0) {
```

```
        if(b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }

    return res;
}

int inv(int a) {
    return Pow(a, mod - 2, mod);
}

int mod_string(string s, int m) {
    int rem = 0;

    for(char c : s) {
        rem = (rem * 10 + c - '0') % m;
    }

    return rem;
}

signed main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    return 0;
}
```